TRANSPORT AND
TELECOMMUNICATION
INSTITUTE

Programming Languages Concepts

Riga 2020

Table of Contents

TEST № 1 3

Task 1..... 3
Task 2..... 4
Task 3..... 5
Task 4..... 5
Task 5..... 6

Appendix 7

Test Example № 1 7

Task 1..... 7
Task 2..... 8
Task 3..... 8
Task 4..... 9
Task 5..... 10

TEST № 1

Task 1.

Represent expressions in prefix and postfix notations, draw an abstract syntax tree for given expressions. Use expressions from Table 1.

Table 1

№	Expression 1	Expression 2
1.	*a * b + c – 2*	*(b / 2 + sqrt ((b / 2) * (b / 2) – a * c)) / a*
2.	*a * (b + c) – 2*	*(a * 2 + c) / d * (a +(b – c * 2)) – tg (f * (2 – a))*
3.	*a * b + (c – 2)*	*(cos ((b / 2) * (b / 2) – a * c)) / a + b / 2*
4.	*a * (b + c – 2)*	*a * 2 + 12 / (d * (a + b – c * 2) – tg (c + a))*
5.	*a * b + c * 2*	*3 + (b / 2 + sqrt ((2 / b) * (b / 2) – a * c)) / a*
6.	*a * (b + c * 2)*	*b / (2 + sqrt ((b / 2) * (b / 2) – a * c)) / a*
7.	*a * (b + c) * 2*	*a * (2 + 12 / d) * (a + b – c * 2) – sqr (f * (c + a))*
8.	*(a * b + c) – 2*	*a / (b / 2 + sqrt (b / 2) * (b / 2) – a * c)*
9.	*(a * b + c) * 2*	*a * 2 + 12 / (d * a + b – c * 2) – sqr (f * (c + a))*
10.	*a – b + c * 2*	*b / 2 + sqrt ((b / 2) * (b / 2) – a * c) / a*
11.	*a *(b + c – 2) – 4*	*a / (b / 2 + sin ((b / 2) * (b / 2) – a * c))*
12.	*2 – a * (b + c)*	*a * 2 + 12 / d * (a + b – c * 2) – sqr (f * (c + a))*
13.	*a * (b + (c – 2))*	*(b + 2 + sqrt ((b / 2) * (b / 2) – a * c)) – a*
14.	*a * (b + c – 2) + t*	*a * 2 / 12 + d * (a + b – c * 2) – sqr (f * (c + a))*
15.	*a * b + c * (t – 3)*	*(b / 2 + sin ((b / 2) – a * c)) * (b / 2)*
16.	*a – (b * c * 2)*	*a * 2 + d * (a + b – c * 2) – cos ((c + a) * f)*
17.	*(a – b * c) + 2*	*(a * 2 + 2 / d * (a +(b – c * 2))) – tg (f * (2 – a))*
18.	*(a * b / c) – 2* d*	*(b / 2 + ((b / 2) * sqrt (b / 2) – a * c)) / a*
19.	*d – (a * b + c) * 2*	*(b / 2 + cos ((b / 2) * a * c)) + a*
20.	*a – b + c + 2 * a*	*a * 12 / d – (a + b – c * 2) – sqr ((c + a) * f)*

Task 2.

Get context-free grammar describing the syntax of the following constructs, see Table 2: NOTE: Check your grammer by pass tree!

Table 2

№	Constructions
1.	Letters and/or digits sequence: a sequence has 0 length or more characters, starts with digit.
2.	Letters and/or digits sequence: a sequence has 0 length or more characters, starts with letter.
3.	Letters and/or digits sequence: a sequence has 0 length or more characters, ends with digit.
4.	Letters and/or digits sequence: a sequence has 0 length or more characters, ends with letter.
5.	Letters and/or digits sequence: a sequence has 0 length or more characters, ends with letter Z.
6.	Letters and/or digits sequence: a sequence has 0 length or more characters, ends with letters WQ.
7.	Letters and/or digits sequence: a sequence has 0 length or more characters, ends with digit 7 or 3.
8.	Letters and/or digits sequence: a sequence has 0 length or more characters, starts with letter N or F.
9.	Letters and/or digits sequence: a sequence has 0 length or more characters, starts with digit 5.
10.	Letters and/or digits sequence: a sequence has 0 length or more characters, starts with digit 8 or with letter C.
11.	Letters and/or digits sequence: a sequence has 0 length or more characters, starts with letters LN.
12.	Letters and/or digits sequence: a sequence has 0 length or more characters, starts with digit 8 or 9.
13.	Letters and/or digits sequence: a sequence has 0 length or more characters, ends with letters EM.
14.	Letters and/or digits sequence: a sequence has 0 length or more characters, starts with digit 4 or 3.
15.	Letters and/or digits sequence: a sequence has 0 length or more characters, ends with letters ED.
16.	Letters and/or digits sequence: a sequence has 0 length or more characters, ends with number 8 or letter M.
17.	Letters and/or digits sequence: a sequence has 0 length or more characters, starts with number 3 or letter S or M .
18.	Letters and/or digits sequence: a sequence has 0 length or more characters, ends with number 6 or letter H or S .
19.	Letters and/or digits sequence: a sequence has 0 length or more characters, starts with number 4 or 3 or letter G.
20.	Letters and/or digits sequence: a sequence has 0 length or more characters, ends with number 3 or letter J or Q.

Task 3.

Using the grammar, show a parse tree and a leftmost derivation, see Table3.

< real_namber> ***::=*** *< integer_part>* **.** *< fractional>*
< integer_part> ***::=*** *< digit > | < integer_part> < digit >*
< fractional> ***::=*** *< digit > | < digit > < fractional>*
< digit > ***::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9***

Table 3

№	String	№	String	№	String	№	String
1.	*1.23456*	**6.**	*9.87654*	**11.**	*56743.2*	**16.**	*0.77734*
2.	*12.3456*	**7.**	*98.7654*	**12.**	*5674.32*	**17.**	*77734.0*
3.	*123.456*	**8.**	*987.654*	**13.**	*567.432*	**18.**	*77.7340*
4.	*1234.56*	**9.**	*9876.54*	**14.**	*56.7432*	**19.**	*777.340*
5.	*12345.6*	**10.**	*98765.4*	**15.**	*5.67432*	**20.**	*7773.40*

Task 4.

Using the grammar, show a parse tree for given expressions in Table4.

<E> ***::=*** *<E>* **+** *<T> | <E>* **–** *<T> | <T>*
<T> ***::=*** *<T>* ***** *<F> | <T>* **/** *<F> | <F>*
<F> ***::= number | name | (<E>)***

Table 4

№	Expression 1	Expression 2
1.	*(2 + a) * 5*	*(2 + (a * b – a / (b + 1)) – 13) * a – c*
2.	*(2 + 3 * d) – 4*	*(2 + (a * b – a / (b + 1)) – 13 * a) – c*
3.	*4 – (b + 3) * 5*	*2 + (a * b – a / (b + 1)) – 13 * a – c*
4.	*7 / (2 + 3)*	*2 + a – (b * a / b) – ((13 * a) – c)*
5.	*7 / (k + 3) * 5*	*(12 + (a – b * a / b) – 13 * a) – c*
6.	*7 * 2 + x*	*12 +((a – b * a / b) – 13 * a) – c)*
7.	*(2 + 3) * 5 – a*	*(12 + (a – b * a / b) – 13) * a – c*
8.	*(2 * 3 + 5) – c*	*(2 * (a – b * a / b) – 13) * a – c*
9.	*(2 + 3) + 5 – a*	*(12 * (a – (b * a) / b) – 13) * a – c*
10.	*a * (2 + 3) * 5*	*(12 * (a – b * a / b) – 13) * (a – c)*
11.	*(2 + a) * 5 – c*	*12 * ((a – (b * a / b) – 13) * a) – c*
12.	*(c * 3 – d) – 4*	*2 + a – (b * a / b) – (13 * a) – c*
13.	*4 / (b + 3) + 5*	*2 + a – ((b * a / b) – 13) * a – c*
14.	*7 – (2 * c) / к*	*(a – b * (a / b) – 13) * (a – c) + 2*
15.	*7 / (k + 3 * a) – 5*	*(a – b * a / b – 13) * (a – c + 2)*
16.	*c – 7 * 2 + x*	*(a – (b * a / b) – 13) * a – c + 2*
17.	*a + (2 + c) * 5*	*2 + (a – (b * a / b) – (13 * a)) – c*
18.	*7 / (2 * 3 + 5) – c*	*12 * (a – (b * a / b) – 13) * a – c*
19.	*(2 + 3) + 5 * a*	*(12 * (a – b) * a) / b – 13 * a – c*
20.	*(2 + 3) * c – 5*	*(12 * (a – (b * a / b)) – 13 * a) – c*

Task 5.

Using the grammar for Pascal language, show a parse tree for the given constructions from Table 5:

< *S* > ::= < *id* > **:=** < *expression* >
| **if** < *expression* > **then** < *S* >
| **if** < *expression* > **then** < *S* > **else** < *S* >
| **while** < *expression* > **do** < *S* >
| **begin** < *SL* > **end**
< *SL* > ::= < *S* > | < *S* > **;** < *SL* >

Table 5

№	Language constructs
1.	***while*** <expression> ***do begin*** <id> ***:=*** <expression>; <id> ***:=*** <expression> ***end***
2.	***if*** <expression> ***then begin*** <id> ***:=*** <expression> ***end***
3.	***while*** <expression> ***do*** ***if*** <expression> ***then*** <id> ***:=*** <expression>
4.	***if*** <expression> ***then*** <id> ***:=*** <expression> ***else begin*** <id> ***:=*** <expression> ***end***
5.	***while*** <expression> ***do*** ***while*** <expression> ***do*** <id> ***:=*** <expression>
6.	***begin*** <id> ***:=*** <expression>***;*** <id> ***:=*** <expression> ***end***
7.	***if*** <expression> ***then*** ***while*** <expression> ***do begin*** <id> ***:=*** <expression> ***end***
8.	***if*** <expression> ***then begin*** <id> ***:=*** <expression> ***end*** ***else*** <id> ***:=*** <expression>
9.	***if*** <expression> ***then*** <id> ***:=*** <expression>
10.	***begin*** <id> ***:=*** <expression> ***end***
11.	***while*** <expression> ***do begin*** <id> ***:=*** <expression> ***end***
12.	***begin if*** <expression> ***then*** <id> ***:=*** <expression> ***end***
13.	***begin while*** <expression> ***do*** <id> ***:=*** <expression>; ***if*** <expression> ***then*** <id> ***:=*** <expression> ***end***
14.	***if*** <expression> ***then*** <id> ***:=*** <expression> ***else*** <id> ***:=*** <expression>
15.	***begin if*** <expression> ***then*** <id> ***:=*** <expression>; ***while*** <expression> ***do*** <id> ***:=*** <expression> ***end***
16.	***begin while*** <expression> ***do*** <id> ***:=*** <expression> ***end***
17.	***if*** <expression> ***then*** <id> ***:=*** <expression> ***else if*** <expression> ***then*** <id> ***:=*** <expression>
18.	***if*** <expression> ***then*** ***if*** <expression> ***then*** <id> ***:=*** <expression>
19.	***while*** <expression> ***do*** <id> ***:=*** <expression>
20.	***while*** <expression> ***do begin*** <id> ***:=*** <expression>; <id> ***:=*** <expression> ***end***

Appendix

Test Example № 1

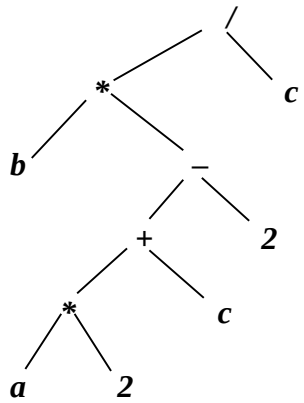
Task 1.

Represent expressions in prefix and postfix notations, draw an abstract syntax tree for given expressions

Expression 1 :

(b * (a * 2 + c – 2))/ c

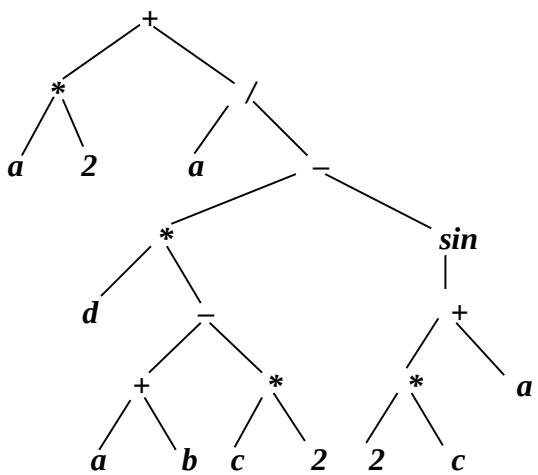
- prefix notation */ * b – + * a 2 c 2 c*
- postfix notation *b a 2 * c + 2 – * c /*
- abstruct syntax tree



Expression 2 :

a * 2 + a / (d * (a + b – c * 2) – sin (2 * c + a))

- prefix notation *+ * a 2 / a – * d – + a b * c 2 sin + * 2 c a*
- postfix notation *a 2 * a d a b + c 2 * – * 2 c * a + sin – / +*
- abstruct syntax tree



Task 2.

Write a context free grammar for the language consisting of strings that have *n* copies of the letter a followed by the same number of copies of the letter b, where *n* > 0.

<u>Desision:</u>

<S>::= a <S>b | ab

Task 3.

Using the grammar, show a parse tree and a leftmost derivation for string 347.45:

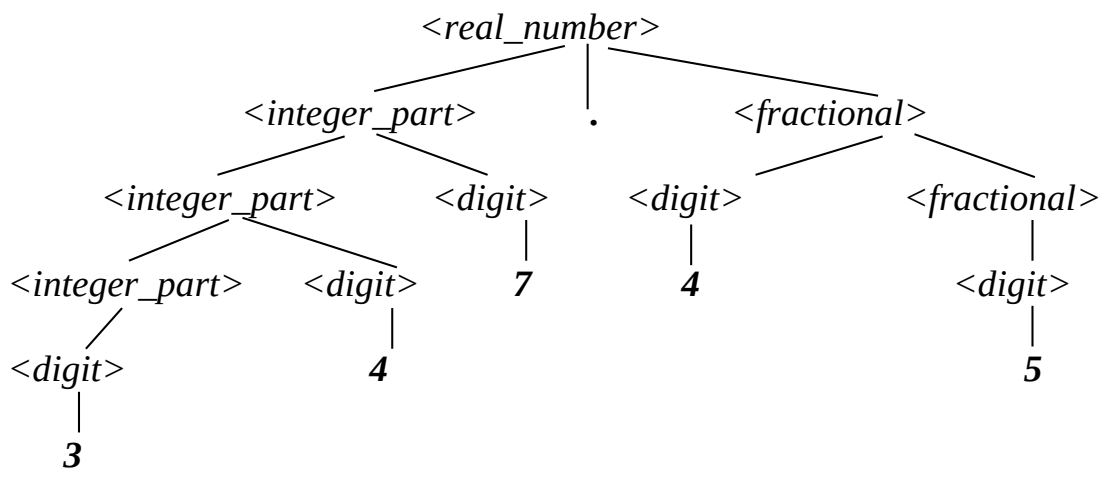
<real_number> ***::=*** *<integer_part>* **.** *<fractional>*
<integer_part> ***::=*** *<digit>* | *<integer_part>* *<digit>*
<fractional> ***::=*** *<digit>* | *<digit>* *<fractional>*
<digit> ***::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9***

<u>Derivation:</u>

<real_number> ⇒
<integer_part> **.** *<fractional>* ⇒
<integer_part> *<digit>* **.** *<fractional>* ⇒
<integer_part> *<digit>* *<digit>* **.** *<fractional>* ⇒

<digit> *<digit>* *<digit>* **.** *<fractional>* ⇒
3 *<digit>* *<digit>* **.** *<fractional>* ⇒
34 *<digit>* **.** *<fractional>* ⇒
347 **.** *<fractional>* ⇒
347 **.** *<digit>* *<fractional>* ⇒
347.4 *<fractional>* ⇒
347.4 *<digit>* ⇒
347.45

<u>Abstract syntax tree:</u>



Task 4.

Using the grammar, show a parse tree for given expressions 1&2:

<E> **::=** <E> **+** <T> | <E> **–** <T> | <T>

<T> **::=** <T> ***** <F> | <T> **/** <F> | <F>

<F> **::=** ***number*** | ***name*** | **(** <E> **)**

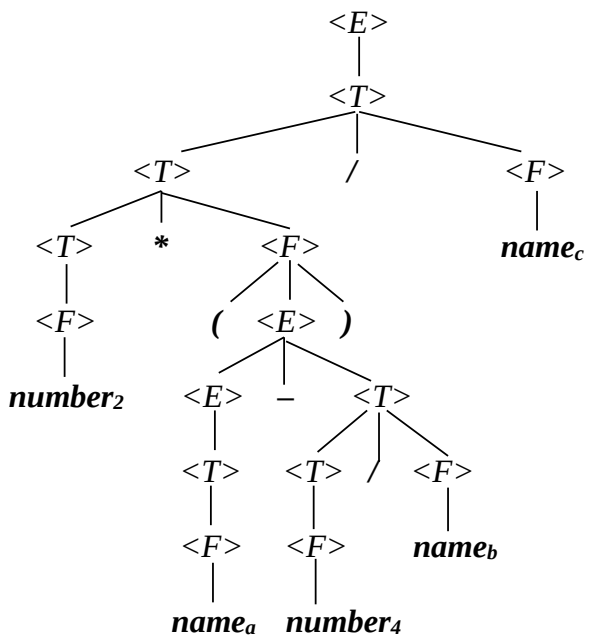
<u>*Expression 1 :*</u>

2 * (a – 4 / b) / c

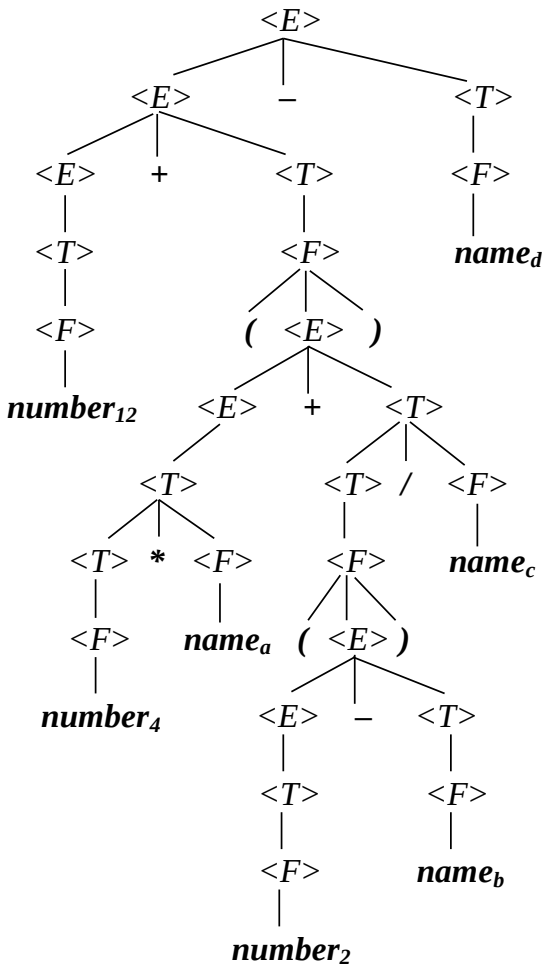
<u>*Expression 2 :*</u>

12 + (4 * a + (2 – b) / c) – d

<u>Abstract syntax tree for expression 1:</u> ***2 * (a – 4 / b) / c***



Abstract syntax tree for expression 2: ***12 + (4 * a + (2 – b) / c) – d***



Task 5.

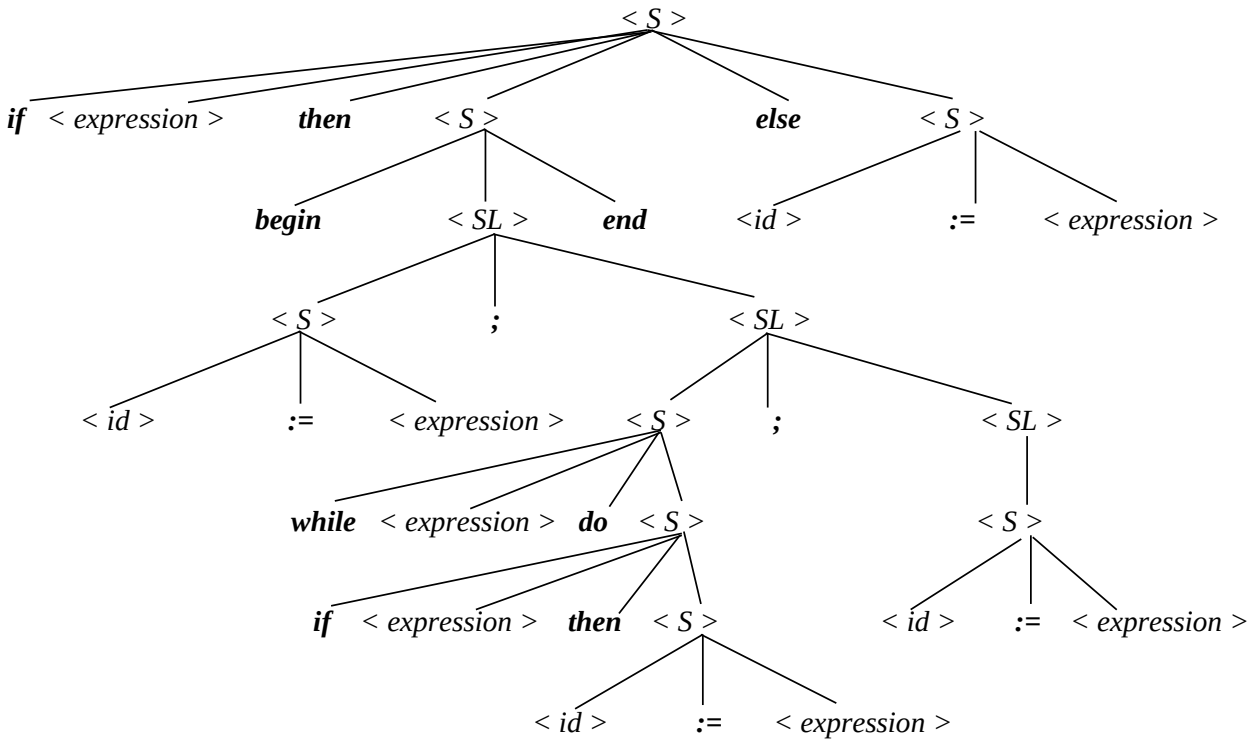
Using the grammar for Pascal language, show a parse tree for the given constructions

< S > ::= < id > ***:=*** *< expression >*
| ***if*** *< expression >* ***then*** *< S >*
| ***if*** *< expression >* ***then*** *< S >* ***else*** *< S >*
| ***while*** *< expression >* ***do*** *< S >*
| ***begin*** *< SL >* ***end***
< SL > ::= < S > | *< S >* ***;*** *< SL >*

Construction:

```
if < expression > then
    begin
        < id > := < expression >;
        while < expression > do
            if < expression > then
                < id > := < expression >;
        < id > := < expression >
    end
    else < id > := < expression >
```

Parse tree:


< S >
if
< expression >
then
< S >
else
< S >
begin
< SL >
end
<id >
:=
< expression >
< S >
;
< SL >
< id >
:=
< expression >
< S >
;
< SL >
while
< expression >
do
< S >
< S >
if
< expression >
then
< S >
< id >
:=
< expression >
< id >
:=
< expression >